

Streamlining geoscience data analysis with an LLM-driven workflow

Jiyyin Zhang, Cory Clairmont, Xiang Que, Wenjia Li, Weilin Chen, Chenhao Li, Xiaogang Ma ^{*} 

Department of Computer Science, University of Idaho, Moscow, ID, 83842, USA

ARTICLE INFO

Keywords:

Large language model
AI agent
Prompt engineering
Geoscience data analysis
Mindat

ABSTRACT

Large Language Models (LLMs) have made significant advancements in natural language processing and human-like response generation. However, training and fine-tuning an LLM to fit the strict requirements in the scope of academic research, such as geoscience, still requires significant computational resources and human expert alignment to ensure the quality and reliability of the generated content. The challenges highlight the need for a more flexible and reliable LLM workflow to meet domain-specific analysis needs. This study proposes an LLM-driven workflow that addresses the challenges of utilizing LLMs in geoscience data analysis. The work was built upon the open data API (application programming interface) of Mindat, one of the largest databases in mineralogy. We designed and developed an open-source LLM-driven workflow that processes natural language requests and automatically utilizes the Mindat API, mineral co-occurrence network analysis, and locality distribution heat map visualization to conduct geoscience data analysis tasks. Using prompt engineering techniques, we developed a supervisor-based agentic framework that enables LLM agents to not only interpret context information but also autonomously addressing complex geoscience analysis tasks, bridging the gap between automated workflows and human expertise. This agentic design emphasizes autonomy, allowing the workflow to adapt seamlessly to future advancements in LLM capabilities without requiring additional fine-tuning or domain-specific embedding. By providing the comprehensive context of the task in the workflow and the professional tool, we ensure the quality of LLM-generated content without the need to embed geoscience knowledge into LLMs through fine-tuning or human alignment. Our approach integrates LLMs into geoscience data analysis, addressing the need for specialized tools while reducing the learning curve through LLM-driven interactions between users and APIs. This streamlined workflow enhances the efficiency of exploratory data analysis, as demonstrated by the several use cases presented. In our future work we will explore the scalability of this workflow through the integration of additional agents and diverse geoscience data sources.

1. Introduction

The recent outcomes in generative Large Language Models (LLMs) manifest significant advancement in natural language processing tasks and lead to extensive applications of LLM in different fields (Naveed et al., 2023; Xi et al., 2023). Among these, OpenAI's ChatGPT, as a state-of-the-art LLM product, has gained intensive attention for its ability to generate human-like responses and perform complex tasks, such as code generation and problem-solving, with a broad range of applications (Roumeliotis and Tselikas, 2023). However, the challenges of knowledge cutoff in the training datasets and the random hallucinations still hamper the application of LLMs in complex workflows. Ongoing research of LLM applications focuses on increasing the adaptability and reliability of LLM responses, such as prompt engineering,

retrieval augmented generation, fine-tuning, and human alignment to mitigate errors and hallucinations. The essential idea of improving the LLM-generated content is exposing the model with sufficient relevant task context. Despite the advancements in LLM capabilities, integrating these models in academic fields presents unique challenges. The creative characteristics of LLMs enable them to understand the human language questions and generate responses, while the identical stochastic nature impedes the model's reliability. During the interactions, LLMs might occasionally be confused by the hallucinations which are not present in user inputs or any other context, and thus generate incorrect result that not aligned with real world truth (Zhang et al., 2023; Tonmoy et al., 2024).

Researchers have explored various approaches to address these reliability concerns, including augmenting LLMs with external tools that

^{*} Corresponding author.

E-mail address: max@uidaho.edu (X. Ma).

<https://doi.org/10.1016/j.acags.2024.100218>

Received 17 September 2024; Received in revised form 3 December 2024; Accepted 19 December 2024

Available online 23 December 2024

2590-1974/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

can provide grounded information and extend the LLM applications. Tool utilization with few-to zero-shot instructions, as one of the essential emergent abilities, significantly broadens the potential application of LLM-based agents (Brown et al., 2020; Wei et al., 2022). Moreover, once these LLMs are pre-trained, they can be taught to utilize tools through few-shot prompt engineering without the need for parameter updates (Mann et al., 2020). The few-shot learning enables LLM-based workflows to solve tasks like code understanding (Nam et al., 2024), medical information extraction (Gebreab et al., 2024; Goel et al., 2023), and financial planning (de Zarzà et al., 2023; Yu et al., 2023), just to name a few. In addition to the few-shot instruction using prompt engineering, researchers showcased that fine-tuning the LLMs with API (Application Programming Interface) calls will expedite them to build the capacity to use the API with zero-shot examples (Patil et al., 2023; Schick et al., 2024). Equipping the tool invocation enhances the perceptual ability of the LLMs to perform tasks more effectively. As a result, it provides access to more environmental information, improving response quality and reducing manual intervention requirements.

Regarding the application of LLMs in geoscience, heavy computation resources and meticulously prepared datasets are used to improve the performance to match general proprietary models, with the sacrifice of capabilities in non-domain specific tasks (Lin et al., 2023; Hadid et al., 2024). Still, the quality of the generated responses is not guaranteed. To bridge the gap between general LLM performance and academic-specific demands, AI agents offer a robust solution. AI agents can perceive environmental information, which enables them to map the perceptual contents into actions through a designed pattern and then conduct the actions accordingly (Russell and Norvig, 2021). In previous studies, intelligent agents have been built on various technologies, such as symbolic rules (Liao, 2005), reinforcement learning (Buşoniu et al., 2010; Zhang et al., 2021), transfer learning (Vrancx et al., 2011; Da Silva et al., 2020), and a variety of other approaches that often intersect and integrate multiple technologies (Xi et al., 2023; Chen et al., 2024). In the geoscience data analysis context, while these intelligent agents have demonstrated efficacy in specific tasks, their capabilities are often constrained by the predefined problems and tools within which they operate. However, with the emergence of LLM agents, there is a growing potential to address a broader range of tasks to leverage AI agents' adaptability and performance across diverse circumstances (Ouyang et al., 2022).

This potential has sparked numerous practical applications in the geoscience domain, particularly with recent advances in LLM technology. ChatGPT, as a representative proprietary LLM, demonstrates diverse applications, including domain-specific uses such as MATLAB code generation for geotechnical engineering research, including seepage flow analysis and slope stability analysis (Kim et al., 2024). Additionally, LLMs have been applied to tasks like seismic risk micro-zoning and simulation parameter recommendation (Wu et al., 2024). These examples illustrate the utility of LLMs as tools in geoscience workflows, where their direct outputs, such as code, are integrated into human-in-the-loop (HITL) processes. These studies primarily focus on evaluating the quality of text or code generated by LLMs rather than creating a streamlined, automated, LLM-driven workflow.

Another significant advancement in the geoscience field involves fine-tuning open-source LLMs using geographic information system (GIS) documents and manually curating large instruction datasets. This approach successfully infuses GIS-specific knowledge into the model, resulting in notable improvements in professional question answering within GIS topics (Zhang et al., 2024c). While fine-tuning domain-specific LLMs holds great promise for professional applications, the substantial effort required to prepare fine-tuning datasets has primarily supported targeted tasks like answering questions rather than facilitating comprehensive geoscience data analysis workflows.

In contrast, the LLM-driven workflow proposed in this study introduces an agentic framework in which LLMs act as autonomous team members. These LLM agents perceive contextual information, make

decisions, and conduct corresponding geoscience data analyses without human intervention. To enable this, we developed the OpenMindat API and data visualization tools specifically for use by LLM agents. This design mitigates risks associated with executing unstable LLM-generated code and enhances workflow autonomy by streamlining the entire data processing pipeline, eliminating the need for HITL contributions.

Compared to fine-tuned LLMs, the proposed LLM-driven workflow leverages meticulously designed prompt instructions and a supervisor-based agentic architecture to encode geoscience knowledge. Once established, the workflow can continuously deliver data analysis services while easily adapting to future advances in LLM technology with minimal effort. This adaptability, combined with the streamlined design, positions our approach as a significant advancement over both HITL implementations and question-answering applications of fine-tuned LLMs.

In this paper, we propose to take advantage of the LLM's adaptive capability on few-shot learning by implementing LLM agents with tools in the geoscience data analysis workflow. We propose to apply a ChatGPT-4o model to handle the geoscience data, utilizing prompt engineering and decision-perceptual instructions to achieve automatic data retrieval and provide preliminary data analysis results. We established an innovative approach in the mineral information field by utilizing LLMs as an interface layer between human users and the open data API of Mindat (a.k.a. OpenMindat API) (Ma et al., 2024; Ralph et al., TBD), one of the largest data portals in geoscience. This approach reduces users' need for proficient programming skills, thus promoting the valuable OpenMindat API to a broader user community in geoscience. Moreover, the developed LLM-driven workflow also has a big potential to be adapted and reused with the APIs of other open geoscience data portals. Although our work is still in its initial stage, it shows the potential of improving the reliability, transparency, and reproducibility of LLM applications in geosciences. We will continue expanding the workflow and use cases and we welcome other users to reuse the work. The remainder of this paper is organized as follows. Section 2 introduces the utilized OpenMindat API and the LLM-driven workflow. Section 3 delineates the workflow use cases in versatile Earth data analysis. Section 4 discusses the observations and thoughts for the constructed workflow and suggests future works. Finally, Section 5 concludes by highlighting the potential of LLM-driven workflows in transforming geoscience data analysis and outlines promising avenues for further research and development in this field.

2. Technical foundation and workflow design

2.1. OpenMindat API and python package

The LLM-driven workflow was designed and developed based on the OpenMindat API (Ma et al., 2024) and Python package (Zhang et al., 2024a), which provide access to the open data service of Mindat. Mindat is a crowd-sourced and expert-curated database of mineral species and their distributions. Thousands of users can contribute datasets to the database, and a group of volunteer scientists monitor the quality of the records and help cleanse them before the public release. The front-end website of Mindat (mindat.org) has attracted about 50 million page views annually in the recent years, underscoring its global impact and significance in the geoscience community. Recently, the OpenMindat API has established the open data service to the key data subjects in Mindat, including mineral species, rocks, localities, and more (Ma et al., 2024; Que et al., 2024). Comparing with the front-end website's role as a human-accessible data source, the API aims to provide a machine-accessible interface for the back-end database of Mindat.

The OpenMindat Python package (see the section Code Availability for the PyPI homepage of the package) was built upon the Mindat API, enabling users to more easily obtain and utilize datasets from Mindat through the Python programming environment. The primary motivation behind the OpenMindat package is to address the increasing demand for

FAIR (Findable, Accessible, Interoperable, and Reusable) (Wilkinson et al., 2016) mineral data. The functions in the package were designed according to the structure of the OpenMindat API. The API itself enhances access to a variety of mineral, rock, and locality data, making it easier for researchers to conduct advanced queries across 20 different data endpoints. As such, the OpenMindat Python package now supports querying over those data endpoints, where each endpoint features a list of customized parameters in mineral attribution, rock classification, locality-related information, and more.

The workflow for using the package involves several key steps, as illustrated in Fig. 1. Users interact with the OpenMindat API through the Python package by first selecting the appropriate data endpoint based on their interests or research topics. They then structure their query needs into a machine-readable dictionary object, specifying parameters such as mineral physical attributes and locality properties. Finally, the API posts the request to the Mindat server and retrieves the records. This package advances the accessibility, interoperability, and reusability of the Mindat open data service by linking the OpenMindat API to popular Python-enabled data science environments such as Jupyter and Google Collab. As a machine-accessible gateway, the OpenMindat Python package not only offers users a streamlined approach to access mineral datasets with customized querying filters, but also facilitates the integration of data retrieval tasks with subsequent data analysis in workflows (Zhang et al., 2024b).

2.2. Design of the LLM-driven workflow

The OpenMindat API and Python package together provide a powerful tool for users to access variety of mineral records from the Mindat database. While it has benefited users with programming expertise, many other potential users are hampered as they have limited skills of computer coding. To bridge this gap, we have designed and developed an LLM-driven workflow that allows users to interact with the OpenMindat API through natural language requests, making it more accessible to a broader user community. Overall, the LLM-driven workflow leverages an agent network to facilitate complex data retrieval and analysis tasks (Fig. 2). At the core of this network is a moderating supervisor agent that organizes the interactions between a user and several specialized LLM action agents, ensuring efficient and accurate execution of user requests.

When a user request is initially received, the supervisor agent needs to determine its validity and suitability for the agent network. If the request is accepted, the supervisor will parse it into several tasks. In the designed working circumstance, the tasks have two major types: data retrieval and data analysis, which are corresponding to the LLM action

agents of collectors and plotters, respectively. Within the supervisor agent, a ChatGPT-4o model with prompt engineering is used to determine the sequence of tasks for the collector and plotter action agents. The prompt instruction for the supervisor consists of three parts: one brief sentence about the supervisor's character and responsibility, a few cases guiding the agent to learn the responding manners, and a list of descriptions of other agents in the workflow.

For the mechanism among the agents, we adopted an open-sourced project for agent status updating and communication (LangChain Inc, 2024). In our proposed workflow, the supervisor LLM agent will react to the initial user request by sending the system instruction and the request to the LLM (in our case, ChatGPT-4o). By parsing the detailed request in the system prompt instruction, the LLM for the supervisor agent will respond with a specific keyword for the next node, including four subsequent action agent names in Fig. 2, plus a "FINISH" option. Consequently, the automatic workflow will then give the flow to the nominated node. The LLM-driven workflow will terminated and returned to the user if the supervisor LLM agent selects the "FINISH" option. Otherwise, i.e., the next node is an action agent. The nominated action agent will be provided with all of the previous workflow processing information, plus its system prompt instruction, to perform actions accordingly, such as calling the OpenMindat API or implementing the visualization tool with the retrieved datasets by the previous agent. Consequently, the execution result(datasets and visualizations) will be returned to the supervisor agent, waiting for the next decision. The detailed instructions and the workflow code are open access (see Appendix A.1 for the prompt details and see section Code Availability for the comprehensive code and dataset requirements to execute this workflow).

For the collector action agents, we have implemented two different data endpoints of 'geomaterial' and 'locality', which are the top two biggest data endpoints in the OpenMindat API. Different data endpoints in the API requires different querying parameters. The data collector agents 'Geomaterial Collector' and 'Locality Collector' are each consisted of three parts: LLM (i.e., ChatGPT-4o), the agent tool for the Mindat dataset retrieval, and the prompt guiding the LLM to use the agent tool. While the LLM for different agents can be identical, the agent tool and the corresponding prompt make each agent specific as they feature different capabilities. The dataset retrieval agent tool uses an output formatter library 'Pydantic' to regulate the response of LLMs. Taking 'Geomaterial Collector' as an example, when querying the 'geomaterial' endpoint in the API, we provide the expected data structure of a dictionary object with valid key names, e.g., using 'elements inc' to mark a list of elements to be included. If the user's initial natural language request involves asking for specific elements to be included,

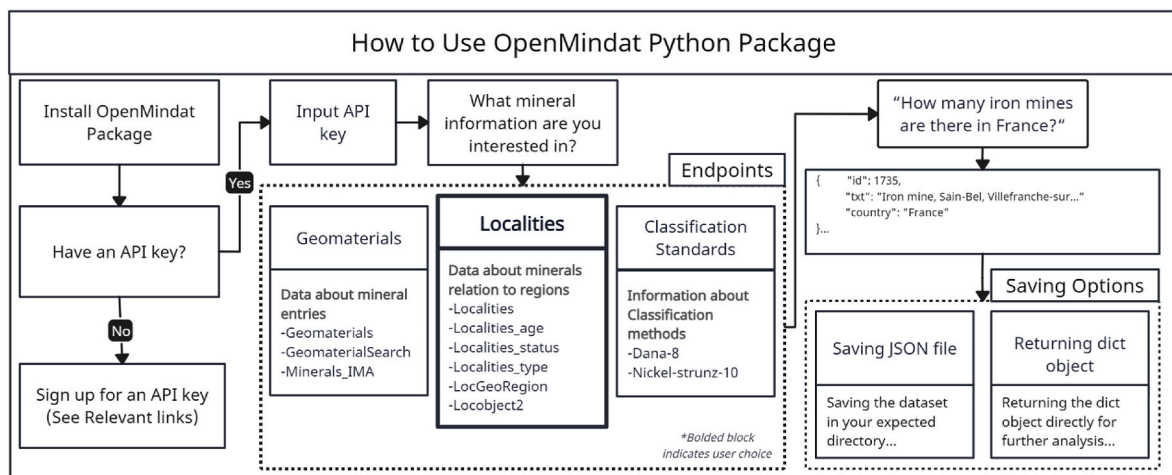


Fig. 1. Workflow of the OpenMindat Python package.

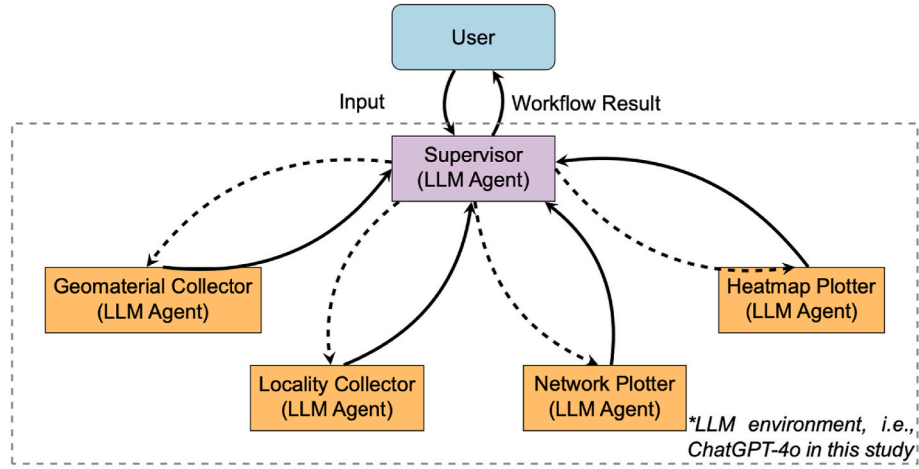


Fig. 2. The conceptual model of the current LLM-driven workflow designed for the OpenMindat API. Note the workflow can be extended by adding more action agents (nodes in orange color in the diagram) into it, such as those similar to the “Geomaterial Collection” and “Network Plotter”.

the LLM will fill the corresponding values to the dictionary keys. Then, the created dictionary object will be passed to the tool functions, i.e., functions in the OpenMindat Python package, to retrieve the data using the arguments in the dictionary object. The ‘Locality Collector’ works similarly to the ‘Geomaterial Collector’, with a focus on retrieving locality data using the ‘locality’ endpoint in the API. Besides the data collector agents, the LLM-driven workflow has two plotter agents playing the role of data analysis and visualization. The ‘Network Plotter’ is used to plot the dynamic network diagram for mineral species, while the ‘Heatmap Plotter’ renders heatmaps for the locality records from the OpenMindat API. More details of them can be seen through the use cases in Section 3.

The functionality of the prompt for the collector and plotter action agents should be highlighted here. The prompt includes three parts: basic role description, the toolkit explanation, followed with a few exemplar cases. For instance, the prompt for the ‘Geomaterial Collector’ action agent guides the LLM to use the OpenMindat Python package to retrieve data from the ‘geomaterial’ endpoint. It defines the agent’s role as a data collector, specifying the tool’s use for gathering data needed for the data analysis and visualization functions in the subsequent plotter agents, while also directing the agent to handle user requests involving locality data by referring them to the ‘Locality Collector’ node. Moreover, the prompt provides a list of pre-defined examples to ensure the LLM consistently performs the correct actions based on user requests, thereby facilitating accurate and efficient data retrieval.

2.3. The multi-turn feature of the workflow

It is noteworthy that the moderating supervisor agent is designed to handle multi-turn reasoning and planning strategies, which are essential for addressing intricate data analysis requests. We can have a closer look at the steps in the workflow to better understand how it is implemented. For instance, a user request such as “Plot the network of element distribution for iron minerals” is executed as steps shown in Fig. 3.

The supervisor agent plays a central role in managing the multi-turn workflow by planning the tasks in a sequence, allocating them to the action agents, monitoring the processing progress, and terminating the workflow when the request is fulfilled. In Fig. 3, the process of an action agent with tool (i.e., collector or plotter) can be represented as:

$$\mathcal{F}(\mathcal{M}_u, \mathcal{M}_s, \mathcal{T}_f, \mathcal{T}_p) = \{\mathcal{A}, \mathcal{T}_a, \mathcal{M}_r\} \quad (1)$$

Here, F is the LLM in the action agent. The parameters in the parenthesis of F are the context information for action agents, including the user request message $\mathcal{M}_u$, the pre-defined system message $\mathcal{M}_s$, the tool function $\mathcal{T}_f$, and the tool parameter template $\mathcal{T}_p$ as input. The context information parameters are then maps them into a set of agent actions $\mathcal{A}$ (e.g., call a tool and return to the supervisor agent), the tool invocation arguments $\mathcal{T}_a$, and the generated message $\mathcal{M}_r$ for the supervisor agent. In Fig. 3 the user request message $\mathcal{M}_u$ is “Plot the network of element distribution for iron minerals”. The system message $\mathcal{M}_s$ is a set of guidelines (i.e., the prompt mentioned in section 2.2) provided to the LLM, defining its role and behavior to ensure appropriate interactions and consistent

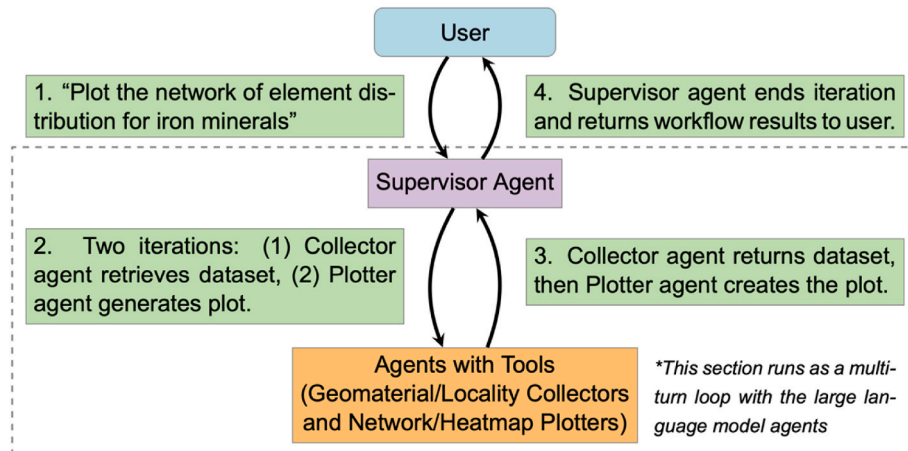


Fig. 3. Steps in the multi-turn LLM-Driven workflow illustrated with an example.

outputs. The M_s includes three parts: the action agent's role as a team member in the workflow, the tool manual for retrieving datasets or creating plots, and exemplary input and output. M_s and M_u work together to guide the agent in correctly understanding the user input and the tasks. The tool function T_f refers to the preset tools equipped to the agent, including the OpenMindat Python package functions for dataset retrieval and the customized visualization functions for plotting network and heatmap diagrams. To invoke the preset tool functions T_f , the LLM in the agent is taught by the tool parameter template T_p to generate structured output corresponding to the task request. For example, the template for calling functions in the OpenMindat Python package is a dictionary object consisting of an array of mineral key-value pairs about the mineral hardness, crystal system information, chemical composition of elements, and more. Consequently, the tool invocation arguments T_a are generated and passed to the tool function T_f to execute an action A , followed by a message M_r about the workflow status, telling the supervisor agent whether the task is successfully fulfilled or needs further attention.

The multi-turn LLM-driven workflow can be delineated as an iteration of LLM interactions, in which the supervisor agent observes the execution status from former processing steps and determines the following actions based on the progress. The process of the supervisor agent can be represented as:

$$\mathcal{F}(\mathcal{M}_u, \mathcal{M}_s, R_t, \mathcal{M}_{t=1}^{n-1}) = \{\mathcal{A}_n, \mathcal{M}_n\} \quad (2)$$

In Equation (2), F is the LLM in the supervisor agent. The input includes four parts of context information. M_u is the message in the user request. M_s is the system message for the supervisor agent, in which the supervisor's role in the workflow is explained and a series of routing examples are provided to guide the appropriate response to handle or reject the user requests. In addition, a comprehensive description of the action agents with tools in the workflow is provided in the system message as well to support the supervisor agent's decision making. Another input is a list of valid routes R_t enumerating the agent names plus one FINISH option. The FINISH option is called when the supervisor agent finds the user request is fulfilled, such as receiving a message from the 'Network Plotter' action agent saying that the mineral network plot is successfully generated. Moreover, accessing the messages from other agents is essential for the supervisor agent to assess the workflow status and supporting the next decision. Here, $\mathcal{M}_{t=1}^{n-1}$ represents all the agent-generated messages from prior rounds. The output here is the action A_n from the list in R_t and the message M_n for the current round n .

3. Use cases

The developed LLM-driven workflow addresses the technical challenges of data retrieval faced by mineral researchers in utilizing the OpenMindat API. Moreover, the workflow can be scaled up with its flexibility in implementing various data analysis and visualization tools in the Python language. In this section, we present three use cases built in our experiment, illustrating the workflow's capability to streamline data retrieval and subsequent visualizations, highlighting the enhanced performance of the LLM agent in typical geoscience data analysis tasks.

3.1. Mindat dataset retrieval

The first use case focused on improving the process of retrieving mineral datasets from Mindat. The OpenMindat API contains about 20 data endpoints, covering different data subjects such as mineral species approved by the International Mineralogical Association (IMA), rock classification, locality records, and more. In the conventional workflow, a user who expects to download a specific dataset from Mindat should follow a few steps: First, interpret the data request, identify the data subjects, and determine the API data endpoint accordingly. Second, look up the OpenMindat API documentation to find the proper parameters for

data query and retrieval. Third, load the OpenMindat Python package and write Python code to retrieve the desired dataset. Users with abundant experience of Python can even write their own functions to query the API directly without using the OpenMindat Python package.

In the LLM-driven workflow, this complex process is simplified through the automation of several key steps. The workflow is divided into three components: user request interpreting, supervisor agent planning, and action agent processing. For instance, in our work the initial user input "Give me the minerals that have cobalt but not chlorine, in the Orthorhombic, Monoclinic, and Triclinic crystal systems" was automatically interpreted by the supervisor agent as a mineral dataset retrieval request, then it planned the tasks and called the corresponding action agents. In this example, the only action agent was the 'Geomaterial Collector', who followed the prompt engineering instruction and Pydantic output formatter to convert the request into a formatted Python dictionary object with API querying parameters of {'crystal system': ['Orthorhombic', 'Monoclinic', 'Triclinic'], 'elements inc': 'Co', 'elements exc': 'Cl'}. The 'Geomaterial Collector' agent then made the call to API through functions in the OpenMindat Python package to get the data. When we ran the experiment in September 2024, a list of 55 cobalt minerals were returned.

Similarly, another user request "Give me the mineral locality records for Brazil" will be interpreted into {'country': 'Brazil'} which is recognizable to the API endpoint 'locality', and the workflow will return all the locality records from the country Brazil. When the experiment was run in September 2024, the workflow resulted 2440 locality records.

Overall, the proposed workflow demonstrated how the LLM-driven workflow could simplify the OpenMindat API usage by interpreting natural language requests into machine-readable parameters and conducting API queries accordingly. At the current stage, our LLM-driven workflow is capable of parsing and interpreting parameters, including IMA flag, minimum and maximum hardness, crystal system, include/exclude chemical element components, and country/region names to adapt a wide range of common API usages, which increases the utility of the OpenMindat API to a broader user community.

3.2. Mineral data query and network visualization

Besides conducting data retrieval with natural language input, the LLM-driven workflow can also include analyzing and visualizing the retrieved dataset by leveraging a variety of other packages and functions in Python. Fig. 4 is a network visualization generated in one of our experiments. The nodes in the network are a list of certain cobalt mineral species that meet the user request, and the links between them show the co-occurrence of those mineral species across known localities in the Mindat database. The size of the nodes represents their frequency of co-occurrence with other mineral species in the network, and the color of the nodes represents the groups in the mineral classification.

The network in Fig. 4 was generated automatically through the LLM-driven workflow. The user request was "Please plot the network of the mineral distribution, which must have cobalt but not chlorine, in the Orthorhombic, Monoclinic, and Triclinic crystal systems". The supervisor agent decomposed the task into two steps: dataset retrieval and subsequent visualization. The 'Geomaterial Collector' in the workflow effectively processed this query by utilizing the OpenMindat API, yielding 55 mineral records that meet the specified filtering conditions. Moreover, as the supervisor agent perceived that the user request was not only about the data retrieval but also the network visualization; after successfully acquiring the dataset, the supervisor agent automatically triggered the next step by assigning the dataset to the 'Network Plotter', the action agent responsible for generating network visualizations using predefined tools. The generated network is an interactive force diagram, and Fig. 4 is a screenshot of it. The code of this use case was shared on GitHub and a link is given in the section Code Availability.

This use case demonstrates the workflow's capability to handle complex requests that include both data retrieval and analysis tasks. We

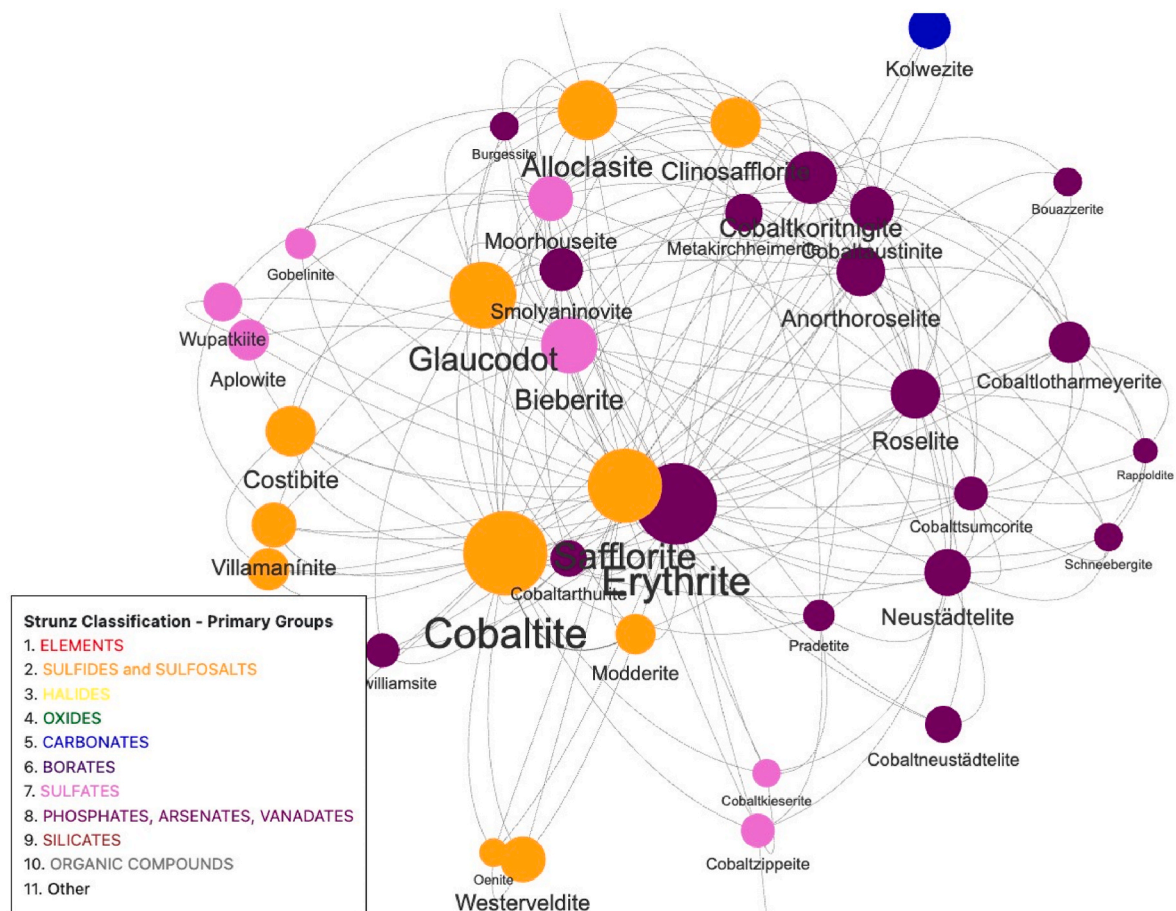


Fig. 4. Co-occurrence network of cobalt minerals generated by the LLM-driven workflow.

can also further enhance the workflow by developing new action agents to link more data analysis and visualization functions in Python, especially those that are widely used by geoscientists.

3.3. Adapting additional LLM agents for scalable locality density heatmap visualization

The previous use cases highlight the LLM-driven workflow's capability in obtaining and analyzing the data from a single data source. However, geoscience researchers used to handle multiple datasets with different sizes, subjects and formats in the real world (Hazen et al., 2019), which poses new requirements for the LLM-driven workflow. In our work we were able to quickly build new agents for data retrieval and analysis to address new scientific topics. Usually, if the data subjects in the user requirement and the Python packages for data analysis and visualization are clearly identified, we only need a few hours to develop the new agents in the LLM-driven workflow.

For example, the 'Locality Collector' and 'Heatmap Plotter' shown in Fig. 2 were developed in our experiment to address an initial user request "Please plot the locality density heatmap for Canada". Then we realized the experiment can be quickly scaled up to include many other countries and regions. Therefore, besides Canada, we randomly picked three other countries to form four individual requests to test the consistency of the LLM-driven workflow in handling this new type of request. In response to each request, the LLM-driven workflow successfully retrieved locality records from the Mindat database, including 6744 records for Canada, 37,455 for France, 10,473 for Germany, and 2194 for South Africa. The locality density maps of those four countries were also generated quickly (Fig. 5). The results show that, once the 'Locality Collector' and 'Heatmap Plotter' agents were successfully

developed with the first example, i.e. Canada, the LLM-driven workflow can address similar requests for other countries or regions fluently by allocating tasks to the corresponding agents in the correct order, without further human interaction needed. The LLM-driven workflow, including all the agents in it, is made open source (see section Code Availability).

This use case demonstrates the flexibility and scalability of the LLM-driven workflow. As geoscience studies often involve multiple data resources and data analysis functions, new agents can be quickly developed and incorporated into the workflow to address the needs of a variety of scientific topics.

4. Discussion

4.1. Observations and thoughts

In this work we have established a framework of supervisor agent and action agents for the LLM applications, and we have successfully connected the framework with the OpenMindat API, the OpenMindat Python package and the broad Python environment to develop a series of use cases. Although the work is still in its initial stage, from the progress so far, we were able to draw a few thoughts.

Reliability of the LLM-driven data analysis workflow. Generative LLMs are renowned for their creativity and adaptability when generating responses to various questions. However, the inherent randomness in LLMs can hinder reliability, particularly when they are applied in more formal contexts such as academic research, where robustness and consistency are valued as well as creativity. Recognizing the concerns about the inherent randomness in LLMs, we are dedicated to constraining them within a specifically designed data analysis workflow. In our proposed workflow, LLMs are used as agents, and all workflow

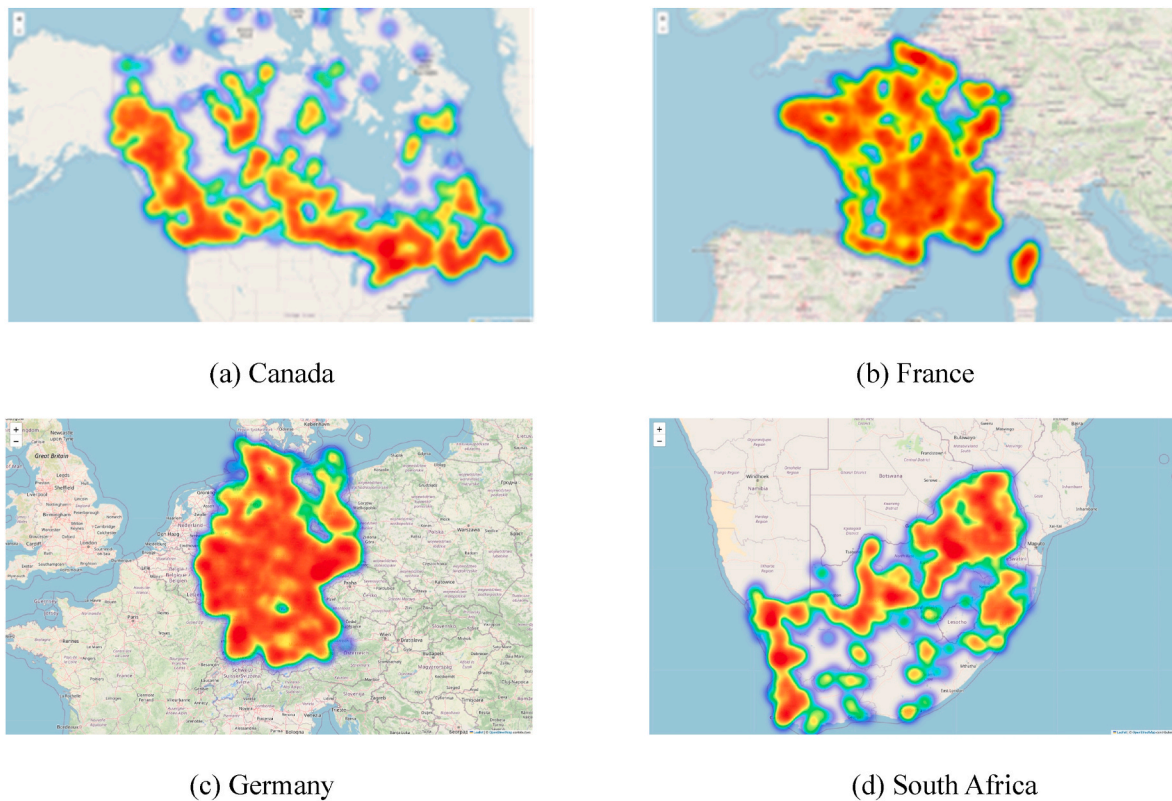


Fig. 5. Locality density heatmaps generated by the LLM-driven workflow.

results are the outcomes of our designed agents with data analyzing tools. This setting effectively ensures that LLM-generated responses are regulated to the designed outcomes, while still taking advantage of LLM's adaptability to parse natural language requests. This method effectively demonstrates the reliability of LLMs in automating complex API queries and function executions. It provides both adaptability and structured control for data analysis tasks and consequently paves the way for many users (especially those with limited programming skills) to apply the data analysis workflow.

Flexibility and scalability of the workflow. Similar to the need for reliability, adaptability is also an essential request with regard to the reproducibility of the LLM applications, and it requires more control over the LLM's application design. Our workflow, built on prompt engineering using the base model ChatGPT-4o, requires no additional fine-tuning and thus highlights the ease of adaptation. The open-source nature of the workflow allows reproduction on platforms such as Jupyter Notebook and Google Colab, making it accessible to a wide range of users. Our successful experience of extending the workflow with new action agents for data retrieval and visualization shows the extensibility of LLM-driven workflow. While the proposed LLM-driven workflow exposes excellent flexibility in addressing geoscience data analytic requests, it can still restrain to a specific scope of utility within the designed fields and intended usages. For example, in the presented heatmap visualization plotting part, the agent is capable of plotting multiple countries and regions in the same picture, while the existing agent setting limited the number of the parsed country names to one, which incurs additional efforts to parse user requests with multiple countries' names. Facing the diverse needs in geoscience research, the workflow can be scaled up to incorporate additional open data portals and APIs, more sophisticated statistical and machine learning methods, and efficient visualization tools. This flexibility opens the door for future expansion across various domains and use cases. We will continue to expand the workflow with more use cases related to the OpenMindat API, and we also welcome other users to adapt the workflow to build

their own applications.

Transparency and reproducibility of LLM actions. The workflow is designed to ensure full transparency, with each step of function calling or decision-making by the LLM agents made accessible for review. This transparency keeps the user informed and involved in the process, like human teamwork. The supervisor agent's decisions and the task execution feedback from the action agents record the stepwise log in the timeline for users to follow what happened when the workflow processed the user requests. Therefore, developers can quickly locate the potential issues and check the reproducibility by comparing the intermediate agent outcomes. In addition, the proposed LLM-driven workflow does not rely on the response generation, which used to be problematic and haunted by hallucination. In the implementation of our workflow, if the tool input is consistent, the workflow will always yield identical results since the LLM is used mainly to interpret the human requests and translate them into function calls, instead of answering the requests directly. The reproducibility of LLM-driven workflow ensures the coherent reliability of the data analysis, showcasing the capability of applying it as an intelligent data analyzing platform where accuracy and robustness are essential, such as geoscience data analysis. As such, the work also provides an exemplary approach to demonstrate how to promote open science (NASEM, 2018) in LLM applications.

4.2. Future work

In the future, we plan to expand the workflow by incorporating additional geoscience open data resources, advanced machine learning methods, and a wider range of visualization tools that are available in the Python environment. We also plan to transition to open LLMs, where fine-tuning strategies will be applied to enhance the efficiency and consistency of the workflow, thereby reducing the reliance on prompt engineering. We assume that the growing amount of LLM agents in the workflow will significantly increase the length of the prompt for the supervisor agent, and the transition to open LLMs will explore

alternative ways to address that issue.

Moreover, as we continuously develop new agents and use cases, comprehensive performance evaluations will be conducted to assess the workflow's effectiveness. Metrics such as accuracy, recall, and pass rates for specific geoscience data analysis tasks will be measured to ensure optimal performance. For the open LLM version of the workflow, we intend to experiment and evaluate how closely it can match the performance of proprietary models, identifying areas where open models excel or fall short. Then, we can propose further development to enhance the performance of the open LLMs in those applications.

In contrast to the proposed workflow, migrating from proprietary to open LLMs presents significant challenges, including the relatively lower general performance of the models and the extensive requirements for preparing fine-tuning datasets. Carefully prepared fine-tuning datasets are essential to bridging the gap between open LLMs and their proprietary counterparts. However, rather than relying on domain documents and publications as training sets, fine-tuning LLM agents in this specific workflow often requires manual dataset creation due to the lack of suitable training sets. To address this challenge, successful practices such as self-instruction pipelines, which leverage LLMs to generate fine-tuning datasets, have shown promise. Nevertheless, these approaches still require substantial human effort for data filtering. Exploring strategies to minimize human involvement and liberate researchers' creativity within an LLM-driven workflow remains the goal of our research.

5. Conclusions

The ability to query and access open data is becoming increasingly crucial for data-intensive geoscience research. However, the needs of computer science knowledge and skills to this process present challenges to many geoscientists, particularly for those with limited skills in programming. The thriving of the LLMs provides potential opportunities to address those challenges. In this work we have focused on enhancing open data accessibility through an LLM-driven workflow. The work was built upon the OpenMindat API and Python package, which provide open data service to Mindat, one of the largest databases in the field of mineralogy. In our research, a framework of supervisor agent and action agents was designed for the LLM-driven workflow, and a list of use cases were successfully developed to demonstrate the utility of it. The workflow's scalability and adaptability provide a new pattern for integrating knowledge in data-driven analysis. Moreover, the transparency of the step-by-step LLM logs ensures interpretability and ease of revision,

facilitating workflow management throughout the data analysis process. Overall, our LLM-driven workflow offers a powerful, flexible, and transparent solution that holds promise for transforming geoscience data analysis.

CRediT authorship contribution statement

Jiyin Zhang: Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Cory Clairmont:** Writing – original draft, Software. **Xiang Que:** Writing – review & editing, Software, Methodology. **Wenjia Li:** Writing – review & editing, Validation. **Weilin Chen:** Writing – review & editing, Validation. **Chenhao Li:** Writing – review & editing, Validation, Conceptualization. **Xiaogang Ma:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Funding acquisition, Conceptualization.

Code Availability

The code developed in this research is available on GitHub: https://github.com/ChuBL/LLM_Driven_Mindat_Workflow. This repository contains all prompt engineering and datasets required to reproduce the results presented in this paper. Some dependencies will require The OpenMindat API key to functionally executed, and the guidelines are provided at: https://www.mindat.org/a/how_to_get_my_mindat_api_key. Information about the OpenMindat Python package is accessible at <https://pypi.org/project/openmindat>.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work presented in the paper was supported by the U.S. National Science Foundation (No. 2126315). The authors thank many fruitful discussions within the Earth Science Information Partners (ESIP) on earlier ideas and progress of this work. We thank two anonymous reviewers for their constructive comments on an earlier version of the manuscript, which helped improve the quality and readability of the presented work.

Appendix A

A.1 System Prompt Configuration

```

1  <|begin_of_text|><|start_header_id|>system<|end_header_id|>
2  You are a supervisor managing a conversation between the workers: {
3  members}. \n
4  Or respond with FINISH if the request is fulfilled. \n
5  Case 1, "User: i want the copper mineral datasets", please go 1.
6  COLLECTOR, 2. FINISH \n
7  Case 2, "User: i want the locality informatino for united states",
8  please go 1. LOCALITY_COLLECTOR, 2. FINISH.\n
9  Case 3, "User: i want the network visualization for copper mineral",
10 please go 1. COLLECTOR, 2. NETWORK_PLOTTER, 3. FINISH.\n
11 Case 4, "User: i want the heatmap of the elements distribution of the
12 ima-approved mineral species with hardness between 3-5, in Hexagonal
13 crystal system, must have oxygen, but without sulfur", please go 1.
14 COLLECTOR, 2. HEATMAP_PLOTTER, 3. FINISH.\n
15 Case 5, "User: I want a heatmap visualization for england", please go
16 1. LOCALITY_COLLECTOR, 2. HEATMAP_PLOTTER, 3. FINISH.\n
17 Case 6, "User: I want to order a coffee", please go 1. FINISH, because
18 such request is beyond our capability\n
19
20 Team members description:
21 - COLLECTOR: the Mindat data collector agent, will collect
22 comprehensive mineral data but without lat long info.\n
23 - LOCALITY_COLLECTOR: the locality data collector agent, will collect
24 locality data with lat long info according to the country name in the
25 request\n
26 - NETWORK_PLOTTER: the network plotter agent, cannot draw without the
27 dataset of collector\n
28 - HEATMAP_PLOTTER: the heatmap plotter agent, cannot draw without the
29 dataset of locality collector, because it need the lat long info.\n

```

Fig. A.1. System prompt for the supervisor agent in the LLM-driven workflow.

Data availability

Data and code are shared on Github and links are given in the paper.

References

- Bușoni, L., Babuška, R., De Schutter, B., 2010. Multi-agent reinforcement learning: an overview. In: Srinivasan, D., Jain, L.C. (Eds.), *Innovations in Multi-Agent Systems and Applications - 1*. Springer, Berlin, pp. 183–221.
- Chen, W., Ma, X., Wang, Z., Li, W., Fan, C., Zhang, J., Que, X., Li, C., 2024. Exploring neuro-symbolic ai applications in geoscience: implications and future directions for mineral prediction. *Earth Science Informatics* 17 (3), 1819–1835.
- Da Silva, F.L., Warnell, G., Costa, A.H.R., Stone, P., 2020. Agents teaching agents: a survey on inter-agent transfer learning. *Aut. Agents Multi-Agent Syst.* 34, 1–17.
- de Zarzà, I., de Curtò, J., Roig, G., Calafate, C.T., 2023. Optimized financial planning: integrating individual and cooperative budgeting models with llm recommendations. *AI* 5 (1), 91–114.
- Gebreab, S.A., Salah, K., Jayaraman, R., ur Rehman, M.H., Ellaham, S., 2024. LLM-based framework for administrative task automation in healthcare. In: 2024 12th International Symposium on Digital Forensics and Security (ISDFS), pp. 1–7.
- Goel, A., Gueta, A., Gilon, O., Liu, C., Erell, S., Nguyen, L.H., Hao, X., Jaber, B., Reddy, S., Kartha, R., Steiner, J., Laish, I., Feder, A., 2023. LLMs accelerate annotation for medical information extraction. In: Heggelmann, S., Parziale, A., Shanmugam, D., Tang, S., Asiedu, M.N., Chang, S., Hartvigsen, T., Singh, H. (Eds.), *Proceedings of the 3rd Machine Learning for Health Symposium*, Volume 225 of *Proceedings of Machine Learning Research*, pp. 82–100.
- Hadid, A., Chakraborty, T., Busby, D., 2024. When geoscience meets generative ai and large language models: foundations, trends, and future challenges. *Expert Syst.* <https://doi.org/10.1111/exsy.13654> (in press).
- Hazen, R.M., Downs, R.T., Eleish, A., Fox, P., Gagne, O., Golden, J.J., Grew, E.S., Hummer, D.R., Hystad, G., Krivovichev, S.V., Li, C., Liu, C., Ma, X., Morrison, S.M., Pan, F., Pires, A.J., Prabhu, A., Ralph, J., Rumyon, S.E., Zhong, H., 2019. Data-driven discovery in mineralogy: recent advances in data resources, analysis, and visualization. *Engineering* 5 (3), 397–405.
- Kim, D., Kim, T., Kim, Y., Byun, Y.H., Yun, T.S., 2024. A ChatGPT-MATLAB framework for numerical modeling in geotechnical engineering applications. *Comput. Geotech.* 169, 106237.
- LangChain Inc, 2024. LangGraph: Build resilient language agents as graphs. <https://github.com/langchain-ai/langgraph>. Accessed: Dec 2024.
- Liao, S.-H., 2005. Expert system methodologies and applications—a decade review from 1995 to 2004. *Expert Syst. Appl.* 28 (1), 93–103.
- Lin, Z., Deng, C., Zhou, L., Zhang, T., Xu, Y., Xu, Y., He, Z., Shi, Y., Dai, B., Song, Y., Zeng, B., 2023. Geogalactica: a scientific large language model in geoscience. *arXiv preprint arXiv:2401.00434*.
- Ma, X., Ralph, J., Zhang, J., Que, X., Prabhu, A., Morrison, S.M., Hazen, R.M., Wyborn, L., Lehnert, K., 2024. Openmindat: open and fair mineralogy data from the Mindat database. *Geoscience Data Journal* 11 (1), 94–104.
- Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., 2020. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* 33, 1877–1901.
- Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., Myers, B., 2024. Using an llm to help with code understanding. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13. Lisbon, Portugal.
- NASEM (National Academies of Sciences, Engineering, and Medicine), 2018. *Open Science by Design: Realizing a Vision for 21st Century Research*. The National Academies Press, Washington, DC, p. 216. <https://doi.org/10.17226/25116>.
- Naveed, H., Khan, A.U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., Mian, A., 2023. A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., 2022. Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst.* 35, 27730–27744.
- Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E., 2023. Gorilla: Large language model connected with massive apis *arXiv preprint arXiv:2305.15334*.

- Que, X., Huang, J., Ralph, J., Zhang, J., Prabhu, A., Morrison, S., Hazen, R., Ma, X., 2024. Using adjacency matrix to explore remarkable associations in big and small mineral data. *Geosci. Front.* 15 (5), 101823. <https://doi.org/10.1016/j.gsf.2024.101823>.
- Ralph, J., Von Bargen, D., Martynov, P., Zhang, J., Que, X., Prabhu, A., Morrison, S., Li, W., Chen, W. and Ma, X., TBD. Mindat.org - the open access mineralogy database to accelerate data-intensive geoscience research. *Am. Mineral.* In Press. DOI:10.2138/am-2024-9486.
- Roumeliotis, K.I., Tselikas, N.D., 2023. ChatGPT and Open-AI models: a preliminary review. *Future Internet* 15 (6), 192.
- Russell, S.J., Norvig, P., 2021. *Artificial Intelligence: A Modern Approach*, fourth ed. Pearson, Hoboken, NJ, p. 2145.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T., 2024. Toolformer: language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* 36, 13.
- Tonmoy, S., Zaman, S., Jain, V., Rani, A., Rawte, V., Chadha, A., Das, A., 2024. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313*.
- Vrancx, P., De Hauwere, Y.M., Nowé, A., 2011. Transfer learning for multi-agent coordination. In: *International Conference on Agents and Artificial Intelligence*, vol. 2, pp. 263–272. <https://doi.org/10.5220/0003185602630272>.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E.H., 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.
- Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J.G., Groth, P., Goble, C., Grethe, J.S., Heringa, J., 't Hoen, P.A.C., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., van Schaik, R., Sansone, S.-A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M. A., Thompson, M., van der Lei, J., van Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B., 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Sci. Data* 3, 160018. <https://doi.org/10.1038/sdata.2016.18>.
- Wu, S., Otake, Y., Mizutani, D., Liu, C., Asano, K., Sato, N., Saito, T., Baba, H., Fukunaga, Y., Higo, Y., Kamura, A., 2024. Future-proofing geotechnics workflows: accelerating problem-solving with large language models. *Georisk* 1–18.
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., 2023. The rise and potential of large language model based agents: a survey. *arXiv preprint arXiv:2309.07864*.
- Yu, X., Chen, Z., Ling, Y., Dong, S., Liu, Z., Lu, Y., 2023. Temporal data meets LLM – explainable financial time series forecasting. *arXiv preprint arXiv:2306.11025*.
- Zhang, J., Clairmont, C., Ma, X., 2024a. OpenMindat Python package. <https://github.com/ChuBL/OpenMindat>. (Accessed 25 April 2024).
- Zhang, J., Que, X., Madhikarmi, B., Hazen, R.M., Ralph, J., Prabhu, A., Morrison, S.M., Ma, X., 2024b. Using a 3d heat map to explore the diverse correlations among elements and mineral species. *Applied Computing and Geosciences* 21, 100154.
- Zhang, Y., Wang, Z., He, Z., Li, J., Mai, G., Lin, J., Wei, C., Yu, W., 2024c. BB-GeoGPT: a framework for learning a large language model for geographic information science. *Inf. Process. Manag.* 61 (5), 103808.
- Zhang, K., Yang, Z., Başar, T., 2021. Multi-agent reinforcement learning: a selective overview of theories and algorithms. In: Vamvoudakis, K.G., Wan, Y., Lewis, F.L., Cansever, D. (Eds.), *Handbook of Reinforcement Learning and Control. Studies in Systems, Decision and Control*, vol. 325. Springer, Cham, pp. 321–384.
- Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., Wang, L., 2023. Siren's song in the ai ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*.